

in Fig. 1. Live image being captured by the camera mounted on the drone during testing is shown in Fig. 2. To make the drone, you will need the components listed in Table 1.

For details regarding drone construction/assembly, propellers, motor rotation, transmitter, and receiver, you may refer to 'Make Your Own Quadcopter Drone' DIY article published in January issue (also available at <https://www.electronicsforu.com>).

Face recognition

At first, an RPi compute module and balenaFin boards having 1GB RAM and 16GB storage, along with LiPo battery, was tried. But it was found to be a little bulky for the drone. So, Raspberry Pi 4 was used because it can be powered from one of the four ESCs used in the drone, which was found to be a better choice.

For face recognition, the RPi 4 and camera should be mounted on F450 frame of the drone (refer Fig. 3). These should be fixed with insulation tape, zip-tied, or tied with a self-locking cable. Since the flight controller board uses only the first ESC pin to power it, the remaining three ESC pins are free. So, use one of the free ESCs to power the RPi 4 board. Cut the positive and negative wires of one of the ESC modules that are connected to the flight controller and then connect them to RPi.

Next, mount the RPi camera module on the drone as shown in Fig. 4.

The code (dronecode1.py) used to recognise the person (known or unknown) in front of the drone is written in Python script. In this code, we need to import three libraries/modules: face_recognition, cv2, and numpy. These are defined in the code as shown in Fig. 5. We need to create different arrays for recognising faces and corresponding names of the persons. Make sure to write the image file name of that person for correct face recognition (refer Fig. 6). Live image of the person (with name) being captured by the camera from the drone is shown in Fig. 7.

Audio alert

In the next part of the code, the program compares the faces being captured by the camera with the array

```
import face_recognition
import cv2
import numpy as np

# This is a demo of running face recognition on live video from
# other example, but it includes some basic performance tweaks
# 1. Process each video frame at 1/4 resolution (though still
# 2. Only detect faces in every other frame of video.

# PLEASE NOTE: This example requires OpenCV (the `cv2` library)
# OpenCV is *not* required to use the face_recognition library.
# specific demo. If you have trouble installing it, try any of

# Get a reference to webcam #0 (the default one)
video_capture = cv2.VideoCapture(0)
```

Fig. 5: Python libraries for face recognition

```
# Family member 1
ashwini_image = face_recognition.load_image_file("ash.jpg")
ashwini_face_encoding = face_recognition.face_encodings(ashwini_image)[0]

# Family member 2
gaurav_image = face_recognition.load_image_file("gaurav.jpg")
gaurav_face_encoding = face_recognition.face_encodings(gaurav_image)[0]

# Create arrays of known face encodings and their names
```

Fig. 6: Names of the known persons included in the code

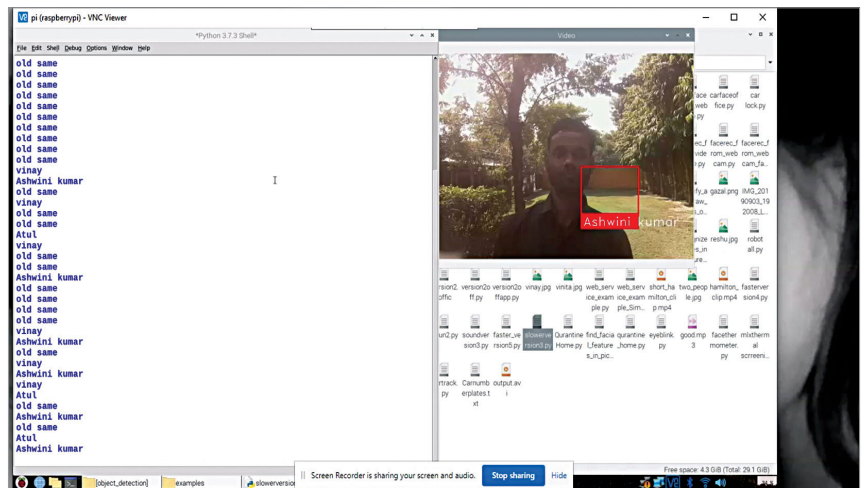


Fig. 7: Live image of a person captured by camera on the drone

of known faces in the database. If the face matches, the code runs the 'espeak.synth ()' synthesiser function to call out the person's name through the speaker connected to the RPi. It can also include an announcement such as "Hello, please don't move," in case a criminal or suspect is to be found with its help. The code with 'espeak.synth ()' function is shown in Fig. 8.

You can connect a digital PAM8403 audio power amplifier and speakers to

the RPi for announcements. Connection diagram of audio amplifier with GPIO, 5V, and GND pins of RPi 4 is shown in Fig. 9 for this purpose.

For crowd detection

The drone can be used for search and rescue operations, especially during floods or earthquake. It can also be used for crowd monitoring during public rallies and gatherings. For this we need to install some libraries, including TensorFlow, in Raspberry Pi.